Betting Partner System

One-page app summary from repository evidence

Pitchsense / Football Intelligence

WHAT IT IS

An AI-driven football prediction and betting-assistant system that produces data-driven match probabilities and Forbidden Fruit slips without relying on bookmaker odds.

It is exposed through a Typer CLI, a FastAPI inference API, and a static Pitchsense dashboard.

WHO IT IS FOR

Not found in repo as a named persona. Repo evidence implies a football betting operator or analyst running daily refresh, drift checks, prediction review, and slip generation.

WHAT IT DOES

- Fetches upcoming fixtures and recent results for supported leagues: PL, PD, SA, BL1, FL1.
- Builds ML-ready features from processed match CSVs: rolling goals, corners, cards, form, rest days, H2H, referee, and weather context where available.
- Trains and routes probabilistic models for goals, corners, and cards with league-specific or global fallback logic.
- Generates match probabilities and confidence output through CLI commands and API endpoints.
- Builds Forbidden Fruit slips from high-probability selections with configurable minimum probability and max selections.
- Applies safety controls: selection gates, drift guardrails, calibration/coverage audits, model-lock checks, and model-health snapshots.
- Serves a web dashboard for Predictions, Slip Builder, Model Health, Drift Monitor, and Terminal views.

HOW IT WORKS

Data sources and files -> CLI ingestion commands -> data/processed/matches/*.csv
FeaturePipeline -> engineered match features -> data/master_features_*.csv
ModelRegistry -> manifest + model artifacts in src/ml/models or MODELS_DIR
Predictor -> sealed CLI prediction loop -> probabilities
Strategies -> SelectionGate, DriftGuardrail, ForbiddenFruitSlipBuilder
Outputs -> Typer CLI, FastAPI endpoints, Pitchsense static dashboard

- API layer: src/api/main.py defines health, predictions, trigger, slips, model info, calibration, and drift endpoints; it mounts /static and /assets and redirects / to /dashboard.
- Runtime support: prediction_cache stores predictions/slips/model health; Sentry hooks capture exceptions; docker-compose includes api, scheduler, model volume, and Postgres services.

HOW TO RUN

Minimal local path from README, user guide, API code, and Dockerfile:

```
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
# Fill API keys: RAPIDAPI_KEY, ODDS_API_KEY,
FOOTBALL_DATA_API_KEY
python -m src.cli fetch-latest-season
python -m src.cli ingest-results
python -m src.cli show-predictions --league PL
```

Dashboard/API option:

```
uvicorn src.api.main:app --host 0.0.0.0 --port 8000
# then open /dashboard
# or: docker compose up -d --build api
```

Note: API startup enforces locked model state unless SKIP_MODEL_LOCK_CHECK=true is set; Render config sets this for deployment.

Repo evidence used

README.md; USER_GUIDE.md; docs/architecture.md; docs/api-reference.md; docs/deployment-guide.md; src/api/main.py; src/predictions/predictor.py; src/features/pipeline.py; src/ml/registry.py; src/static/index.html; Dockerfile; docker-compose.yml; render.yaml; .env.example.